

RapidAPI Data 静态 IO 模型：识别诊断与补充数据路线图

Codex

2026-06-15

Contents

摘要	3
1. 当前识别哪里弱	3
2. 不支持“数据这一商品特点”的地方	4
3. 修正后的经济模型重点	4
3.1 无限供给不是“无成本”	4
3.2 观测数量是下界	4
3.3 试用机制应是漏斗模型	5
4. 最优先补充的数据	6
4.1 RapidAPI analytics 和 revenue analytics	6
4.2 重复快照形成 RapidAPI 面板	6
4.3 历史网页档案：Wayback 和 Common Crawl	7
5. 数据商品专属增强变量	7
5.1 数据源和源平台成本冲击	7
5.2 数据新鲜度、覆盖与 schema	8
5.3 复制外溢与真实使用量 proxy	9
5.4 外部需求和开发者采用	9
6. 具体识别设计	9
6.1 最可行：API 固定效应 + 价格变化事件	9
6.2 强化版：源平台成本冲击 IV	10
6.3 plan-level nonlinear tariff	10

6.4 试用-转化漏斗	10
6.5 排名/曝光准实验	11
7. 数据优先级	11
8. 建议立刻执行的补数方案	11
9. 资料来源	12

摘要

当前静态 IO 模型已经形成闭环，但最弱的地方不是代码或模型形式，而是识别信息不足：价格是当前菜单，订阅数是累计存量，缺少当期成交、调用、收入和 plan-level 选择份额。因此，2SLS 第一阶段弱，供给侧边际成本只能校准反推，不能作为强结构估计。

还需要特别强调三点数据商品特征：第一，企业面对的是近似无限供给的非竞争性商品，不是有产能约束的实体商品；第二，买方可能复制、缓存或把数据传给其他人，所以平台看到的订阅数只是直接买方数量，真实下游使用量更大；第三，试用不是简单的免费 dummy，而是“发现-试用-质量学习-付费转化-套餐选择”的漏斗。

要把研究推进为更强的“数据商品”论文，下一步应补充三类真实可得数据：

1. 动态面板：重复记录 API 价格、套餐、订阅数、排名、评分、质量和文档变化。
2. 真实使用与收入：RapidAPI provider/consumer analytics、revenue analytics 或与卖家合作获得的 API-level/plan-level 调用和收入。
3. 数据商品专属特征：数据源、更新频率、字段覆盖、响应样例、合规/授权说明、源站 API 成本冲击和外部需求冲击。

1. 当前识别哪里弱

模块	当前做法	识别问题	后果
需求价格系数	横截面 OLS/2SLS logit	OLS 价格系数为正； 2SLS 转负，但第一阶段 $F = 3.48$	不能直接把 2SLS 价格系数用于供给 FOC
数量/份额	subscriptions_count + 1	订阅数是累计存量，不是当期销量或调用量	价格和数量时间不匹配
市场规模	inside share 校准为 0.20	外部选择和潜在买家数量未观测	弹性和反事实量级依赖校准
工具变量	rival characteristics、soft/hard limit、overage	很多变量既影响成本也直接影响买方效用	排除限制较强
供给侧	静态 Bertrand 反推 markup/MC	缺少真实成本、收入、平台抽成	边际成本是影子成本，不是会计成本
无限供给	把 API 当差异化产品	数据复制成本近零，产能不是约束	供给模型应解释访问控制和价格歧视，而非产量供给
复制外溢	$q = \text{subscriptions} + 1$	买方可复制/缓存/分享数据	q 是真实使用量下界，福利和需求规模被低估
试用机制	has_free_plan dummy	无 free quota、trial calls、conversion	无法区分试用降低不确定性和免费额度替代付费
plan-level 选择	用 API 层最低价	没有 BASIC/PRO/ULTRA 等计划的购买份额	无法识别非线性价格菜单选择

模块	当前做法	识别问题	后果
数据商品机制	额度、免费入口、文档、评分	缺少数据新鲜度、来源、字段覆盖、合规权利	“数据商品”边际贡献还不够硬

2. 不支持“数据这一产品特点”的地方

当前模型把数据商品部分刻画为“访问权 + 额度 + 超额费 + 声誉信号”，但以下特征还没有被充分观测：

1. 新鲜度：是否实时、每日、每周更新；当前只有文本关键词和 `updatedAt`。
2. 覆盖范围：覆盖多少国家、平台、字段、实体、历史窗口；当前没有结构化 `coverage`。
3. 数据质量：缺失率、重复率、字段一致性、响应稳定性；当前只有平台展示的成功率/延迟/评分。
4. 权利与合规：是否说明数据来源、授权、隐私、`robots/API terms`；当前基本没有结构化变量。
5. 下游用途强度：买家真实调用量、`endpoint` 使用分布、超额费发生频率；当前没有。
6. 非线性合同选择：买家在 `free/basic/pro/ultra/mega` 间如何选择；当前没有 `plan-level share`。
7. 复制外溢：数据下载后是否在组织内复用、缓存、再分发；当前没有 `kappa_j`。
8. 试用转化：免费调用是否转成付费，转化发生在哪个价格计划；当前只有是否存在免费计划。

这些缺口导致论文很容易被读成“API 产品定价研究”，而不是“数据作为商品的市场设计与定价研究”。

3. 修正后的经济模型重点

3.1 无限供给不是“无成本”

数据 API 近似非竞争：同一份数据可以被多个买方同时使用，卖家不需要像实体商品那样扩大产量。因此，供给侧不应强调产能或数量生产，而应强调：

1. 固定成本：数据源获取、清洗、`schema` 维护、接口开发、合规审查。
2. 每次请求成本：服务器、上游 API、带宽、失败请求处理。
3. 访问控制成本：API key、`rate limit`、`quota`、超额计费、风控。
4. 复制外溢成本：买方下载后绕开平台继续传播，卖家无法按真实下游使用收费。

所以供给 FOC 中的 `mc_j` 应解释为“访问权边际影子成本”，而不是复制一份数据的生产成本。

3.2 观测数量是下界

当前模型使用：

$$q_{\text{obs},j} = \text{subscriptions}_j + 1$$

但对数据商品，更合理的是：

```
Q_true,j = kappa_j * q_obs,j  
kappa_j >= 1
```

kappa_j 是复制、缓存、组织内共享或再分发倍率。它可能随数据类型变化：

数据类型	复制外溢可能性	原因
静态表格/参考数据	高	下载后可长期复用
企业线索/联系人数据	高	可进入 CRM 或销售团队共享
文档/文本抽取	中高	输出文件可内部流转
实时价格/金融/天气	较低	新鲜度衰减快，需持续 API 调用
身份校验/地理查询	中	单次查询结果可保存，但新请求仍需调用

如果高价 API 更可能被组织内多人共享，那么 q_obs 对真实使用量的低估与价格相关，会使需求弹性和福利估计偏误。

3.3 试用机制应是漏斗模型

当前 has_free_plan 太粗。更合适的静态漏斗是：

Exposure -> Trial -> Learn quality -> Paid conversion -> Plan choice

可写成：

```
Pr(trial_j = 1)  
  = Lambda(a0 + a1 free_quota_j + a2 search_rank_j  
           + a3 approval_friction_j + x_j a)  
  
signal_ij = true_quality_j + noise_ij  
  
Pr(pay_j = 1 | trial_j = 1)  
  = Lambda(b0 + b1 E[true_quality_j | signal_ij]  
           - b2 price_j + b3 quota_j + b4 overage_j + x_j b)
```

免费计划的经济作用有两面：

1. 学习效应：降低买方对数据质量、覆盖、字段和稳定性的 uncertainty。
2. 替代效应：免费额度过大时，轻度买方不升级付费计划。

因此，仅用 has_free_plan 会混合两种相反机制。真正需要的是 free quota、trial calls、conversion rate 和 paid plan choice。

4. 最优先补充的数据

4.1 RapidAPI analytics 和 revenue analytics

RapidAPI 官方文档说明, consumer analytics 可以查看 API call history, 按 App、API、endpoint 和时间拆分, 并可导出 raw logs。provider analytics/revenue analytics 对 monetized APIs 可查看固定价格收入和 overage 收入, 历史窗口最长到一年。这是最强的数据, 但需要 API consumer/provider 账号权限, 或与若干卖家合作。

可补字段:

字段	层级	识别用途
API calls by day/week	API-endpoint-time	当期需求流量, 替代累计订阅数
calls by plan	API-plan-time	识别套餐选择和 nonlinear tariff
overage calls	API-plan-time	识别边际使用价格和 soft limit
free trial calls	API-plan-time	识别试用强度
free-to-paid conversion	API-plan-user-time	区分学习效应和替代效应
unique apps/users	API-time	识别直接买方数量
calls per app/user	API-time	推断组织内复用或重度使用
fixed revenue	API-time	估计真实价格收入
overage revenue	API-time	分解两部制定价
error/latency by endpoint	endpoint-time	质量冲击和体验品学习
app/user count	API-time	区分少数重度用户和多数轻度用户

识别增益: 可以把需求方程从累计采用模型改为流量/收入面板模型:

$$\log(\text{calls_jt}) = \alpha \log(\text{price_jt}) + \beta \text{quota_jt} + \gamma \text{quality_jt} + \text{API_FE_j} + \text{time_FE_t} + \text{exposure_jt} + \text{epsilon_jt}$$

并进一步估计 plan-level nested logit:

$$\begin{aligned} \text{choice} &= \text{outside} / \text{free} / \text{basic} / \text{pro} / \text{ultra} / \text{mega} \\ \text{utility_jkt} &= \alpha \text{price_jkt} + \beta \text{quota_jkt} + \gamma \text{overage_jkt} \\ &\quad + \theta \text{quality_jt} + \xi \text{jkt} + \text{epsilon_ijkt} \end{aligned}$$

4.2 重复快照形成 RapidAPI 面板

这是最现实、最应该马上做的数据。每周或每天固定记录以下字段:

字段	来源	识别用途
subscriptionsCount	API 详情页/接口	用差分近似新增采用
plan price/quota/overage	pricing tab	捕捉价格和合同变化
free plan quota	pricing/billing limits	识别试用强度

字段	来源	识别用途
approval requirement	pricing tab	识别试用/购买摩擦
search rank by keyword/sort	RapidAPI search	控制曝光，估计排名冲击
rating/votes	API 详情页	声誉学习
avgLatency/success/service level	API 详情页	技术质量变化
updatedAt/readme length	API 详情页	产品维护和信息披露

面板后可以做：

```
Delta log(subscriptions_jt + 1)
= alpha Delta log(price_jt)
+ beta Delta quota_jt
+ gamma Delta rank_jt
+ API_FE_j + week_FE_t + epsilon_jt
```

或者事件研究：

```
new_subscriptions_jt = sum_k beta_k 1[t - price_change_j = k]
+ API_FE_j + week_FE_t + controls_jt + epsilon_jt
```

这比横截面强很多，因为 API 固定效应可以吸收稳定质量差异。

4.3 历史网页档案：Wayback 和 Common Crawl

Internet Archive 的 CDX API 可以查询网页历史抓取记录，Common Crawl 提供免费网页抓取数据和索引。它们可以补足我们开始系统快照之前的历史价格/描述/套餐记录，但对动态 JavaScript 页面覆盖可能不完整。

可补字段：

字段	识别用途
历史 API 页面是否存在	entry/exit timing
历史 pricing 文本	价格变动事件
历史 readme/description	数据商品定位变化
历史 owner website	卖家外部声誉

适合做辅助历史面板，不适合作为唯一数据源。

5. 数据商品专属增强变量

5.1 数据源和源平台成本冲击

对很多 Data API，成本来自上游平台、数据源或抓取难度。可按 API 名称/描述识别 LinkedIn、Amazon、Reddit、X、Google Maps、Instagram、TikTok、Zillow 等源平台，再匹配源平台 API/数据访问政策或价格变化。

真实可用例子：

源平台	可得数据	用途
Reddit	2023 年 API 价格和 rate-limit 变化	上游数据成本冲击
X/Twitter	API pricing/credit/pay-per-usage 规则	社交数据源成本冲击
Google Maps	Maps Platform API 价格、用量和免费额度规则	地理数据成本冲击
AWS	Price List API	云计算/带宽成本控制变量

设计：

```
price_jt = pi cost_shock_source(j,t) + API_FE_j + time_FE_t + eta_jt
log(calls_jt) = alpha predicted_price_jt + controls + FE + epsilon_jt
```

注意：源平台政策变化可能也直接影响下游需求和替代关系，所以最好做受影响/未受影响组的 event study，并检验预趋势。

5.2 数据新鲜度、覆盖与 schema

这些变量更能体现“数据商品”。

变量	获取方式	经济含义
freshness promise	从文档抽取 real-time/daily/live/historical	数据时效性
source platform	文本分类或 endpoint path	数据来源和合规风险
field count	response example/schema 解析	信息维度和覆盖
endpoint count	API 文档解析	产品范围
coverage country/platform	文档 NER/关键词	横向覆盖
auth/compliance words	文档抽取 GDPR/CCPA/official/authorized	合规/权利风险
sample response missingness	免费计划试调用	数据质量
live latency/error	定期试调用	体验品质量
downloadable/static output	响应格式和文档	复制外溢风险
cacheability/freshness decay	文档和重复调用测试	数据是否需要持续访问
terms restriction	文档/法律文本	是否禁止再分发

设计上，这些变量进入 $\mathbf{x}_j$ ，并作为随机系数或 nested logit 的异质性维度。

5.3 复制外溢与真实使用量 proxy

要处理 q_{obs} 低估问题，至少需要构造 κ_j proxy:

proxy	获取方式	含义
calls per subscriber	provider analytics	单个买方是否大量复用
unique API keys/apps	analytics	直接买方数
endpoint response size	试调用或文档样例	单次调用可复制的数据量
static vs real-time	freshness 文本和重复调用	静态数据更易传播
CSV/bulk/export words	文档关键词	下载型数据复制风险更高
CRM/enrichment/lead words	文本分类	组织内共享可能更高
legal terms restriction	文档/approval question	卖家是否主动限制再分发
GitHub mentions	GitHub search	开发者外部采用痕迹

在模型中可以写为:

$$\log(Q_{\text{true},j}) = \log(q_{\text{obs},j}) + \log(\kappa_j)$$
$$\log(\kappa_j) = z_j \rho + \text{error}_j$$

如果没有真实 Q_{true} ，至少要在稳健性中按数据类型设定不同 κ ，报告总使用量和福利区间。

5.4 外部需求和开发者采用

可用外部数据增强市场规模和需求冲击:

数据	来源	用途
keyword interest	Google Trends	市场需求冲击
website traffic	Similarweb/Semrush	卖家声誉、下游需求 proxy
website performance	CrUX BigQuery	卖家/源站质量 proxy
GitHub code search	GitHub REST API	API 被开发者代码引用的采用 proxy
PyPI downloads	PyPI BigQuery	SDK/爬虫/数据工具生态需求
npm downloads	npm downloads API/registry	JS 数据工具生态需求
Stack Exchange questions	Stack Exchange Data Explorer/data dump	开发者问题热度

这些变量不能完全替代真实交易，但可以提供市场需求侧外生变化和产品可见度控制。

6. 具体识别设计

6.1 最可行: API 固定效应 + 价格变化事件

需要数据: 每周 RapidAPI 面板。

```

new_subs_jt = subscriptions_jt - subscriptions_j,t-1
log(1 + new_subs_jt) = alpha log(price_jt)
                      + beta quota_jt
                      + gamma rank_jt
                      + API_FE_j + week_FE_t + epsilon_jt

```

优势：去掉 API 固定质量。

不足：价格变化可能仍由预期需求驱动。

增强：加入 search rank、rating changes、quality changes、readme changes；做 price-change event study 和 pre-trend。

6.2 强化版：源平台成本冲击 IV

需要数据：API 源平台分类 + 源平台 API/数据访问价格变动。

```

log(price_jt) = pi affected_source_j * post_t + API_FE_j + week_FE_t + eta_jt
log(1 + new_subs_jt) = alpha predicted_log_price_jt + FE + controls + epsilon_jt

```

优势：更接近供给侧成本 shifter。

不足：源平台政策也可能改变买方需求和合法可用性，需解释排除限制。

可做 placebo：未涉及该源平台的数据 API 不应同步跳价。

6.3 plan-level nonlinear tariff

需要数据：各 plan 的调用量/订阅数，最好来自 provider analytics。

```

U_ijkt = alpha monthly_fee_jkt + beta quota_jkt + gamma overage_jkt
        + rho recommended_jkt + theta quality_jt + xi_jkt + epsilon_ijkt

```

优势：直接研究数据商品“访问权 + 额度 + 超额费”的合同选择。

这是最能体现边际贡献的方向。

6.4 试用-转化漏斗

需要数据：free plan quota、trial calls、unique trial users、paid conversion、plan choice。

```

Trial_jt = 1[free calls > 0]

```

```

Paid_jt = 1[paid calls or fixed revenue > 0]

```

```

Pr(Trial_jt) = Lambda(free_quota_jt, rank_jt, approval_jt, quality_jt, FE)

```

```

Pr(Paid_jt | Trial_jt)
  = Lambda(price_jt, quota_jt, trial_success_jt, latency_jt,
           freshness_jt, FE)

```

这能区分免费计划的学习效应和替代效应。若免费计划提高 trial 但降低 conversion，需要在反事实中同时模拟两段。

6.5 排名/曝光准实验

需要数据：搜索关键词、排序方式、每日排名。

$$\log(1 + \text{new_subs_jt}) = \text{beta rank_jt} + \text{API_FE_j} + \text{keyword_time_FE} + \text{controls} + \text{epsilon_jt}$$

- 优势：识别平台曝光如何影响采用。
- 不足：排名可能由订阅数/质量内生决定。
- 增强：利用排序规则变化、关键词窗口变化、新 API 冷启动或相邻排名比较。

7. 数据优先级

优先级	数据	可得性	主要解决的问题	推荐程度
1	每日/每周 RapidAPI 面板	我们可直接做	价格-数量时间错配、API FE	最高
1	search rank/exposure 面板	我们可直接做	曝光控制和需求冲击	最高
1	plan/price/quota 变化事件	我们可直接做	非线性合同变化	最高
1	free quota/approval/trial 条款	我们可直接做	试用机制	最高
1	cacheability/freshness schema	文档和测试调用	数据商品差异	最高
2	provider analytics/revenue	需卖家合作	真实调用、收入、overage	极高
2	consumer analytics raw logs	需账号/合作	endpoint-level 使用	极高
2	calls per subscriber/app	需 analytics	复制/复用倍率 proxy	极高
2	source platform cost shocks	公开 + 文本分类	成本侧 IV	高
3	Wayback/Common Crawl 历史页面	公开免费	补历史价格/进入	中高
3	Google Trends/GitHub/PyPI/npm	公开或低成本	外部需求 proxy	中高
4	Similarweb/Semrush	商业付费	卖家/源站流量	中
4	人工试调用/API quality monitor	需要账号和配额	实测质量	中高

8. 建议立刻执行的补数方案

第一阶段，两周内可做：

1. 每天固定抓取 Data 类别 API 详情页、pricing、billing limits。
2. 固定 30-50 个关键词: data, scraper, linkedin, amazon, company, finance, geocode, real estate, people, news 等, 记录 search rank。
3. 对每个 API 抽取源平台、freshness、coverage、schema words、合规词、cacheability、是否 bulk/export。
4. 用 Wayback CDX 对头部 500 个 API 回溯历史页面。
5. 建立 api_id-week 面板, 变量包括 price、quota、free quota、approval、rank、subscriptions、rating、quality、readme。

第二阶段, 一个月内可做:

1. 分类 Reddit/X/Google Maps/Amazon/LinkedIn 等源平台 API。
2. 收集源平台 API 政策/价格变化时间点。
3. 做 price-change event study、rank exposure model、source-cost-shock DiD/IV、free-trial funnel model。
4. 输出一个新版 IO 模型: API FE demand + calibrated supply + stronger counterfactual。

第三阶段, 合作数据:

1. 找 5-10 个 RapidAPI provider, 要一年的 analytics/revenue export。
2. 把调用量和收入对齐到 plan/endpoints。
3. 估计 calls per subscriber、trial-to-paid conversion 和 plan-level nonlinear tariff model, 这是最能体现数据商品机制的版本。

9. 资料来源

- RapidAPI Analytics Overview: <https://docs.rapidapi.com/docs/analytics-overview>
- RapidAPI Provider Analytics: <https://docs.rapidapi.com/docs/provider-analytics>
- RapidAPI Consumer Quick Start / pricing plans: <https://docs.rapidapi.com/docs/consumer-quick-start-guide>
- RapidAPI Monetizing APIs: <https://docs.rapidapi.com/docs/monetizing-your-api-on-rapidapicom>
- Internet Archive Wayback CDX API: https://archive.org/help/wayback_api.php
- Common Crawl Index Server: <https://index.commoncrawl.org/>
- Common Crawl Get Started: <https://commoncrawl.org/get-started>
- GitHub REST API Docs: <https://docs.github.com/en/rest>
- PyPI BigQuery dataset: <https://docs.pyapi.org/api/bigquery/>
- Google Cloud BigQuery public datasets: <https://docs.cloud.google.com/bigquery/public-data>
- Chrome UX Report on BigQuery: <https://developer.chrome.com/docs/crux/bigquery>
- Similarweb API docs: <https://developers.similarweb.com/>
- Semrush API docs: <https://developer.semrush.com/api/seo/overview/>
- Reddit API change discussion: https://www.reddit.com/r/reddit/comments/145bram/addressing_the_community_about_changes_to_our_api/
- Reddit Data API Terms: <https://redditinc.com/policies/data-api-terms>
- X API pricing: <https://docs.x.com/x-api/getting-started/pricing>
- Google Maps Platform pricing: <https://developers.google.com/maps/billing-and-pricing/pricing>
- AWS Price List API: <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/price-changes.html>